

Asymmetric Laplace Isomorphic Conic Encoding Technical Documentation



Figure 1: Result after denoising with the ALICE encoder and an SVGF-like denoiser, 5 bounces

Contents

Preface	4
Background	4
Purpose	4
Modeling	4
First Principles	4
Operator Derivation	6
Direct Derivation	6
Cone Closure Proof	6
Linearity	6
Uniqueness Proof	6
Probabilistic Origin of the Cone Constraint	7
Jensen' s Inequality Guarantee	7
Reinterpretation of Boundary States	7
Statistical Meaning of the Jensen Gap	7
Algorithmic Significance	7
Statistical Model	8
Runtime Closure	8
Second-Order Statistics and Dual Parameters	8
Lambert Query	9
EON Query	9
GLSL Interface	9
Denoising Pipeline Implementation	10
Pipeline Overview	10
Unified Diffuse Buffer	12
Variance Pre-Filtering (swap2)	13
Design Motivation	13
Cooperative Shared-Memory Loading	13
3-Sigma Energy Clamping	14
5×5 Geometry-Aware Variance Filtering	14
Gaussian Curvature Edge Marking (Optional, Non-Essential)	14
Spatial Filter (300 / 300_cs)	15
À-Trous Wavelet Decomposition Architecture	15
Compute Shader Shared-Memory Optimization	15
Tri-Lateral Weight Function	16
Critical Note on the Asymmetric σ^2 Choice	17
Parameter Sensitivity of the Variance Floor	17
Sample Accumulation in ALICE Augmented Space	18
Rotation Jitter	19
Special Handling of the Final Pass	19
Buffer Swap (swap3)	20
Data Flow Summary	20
Naming and Scope	21
Screen-Space Path Rebuilding Importance Sampling	22
Physical Motivation and Prior Distribution	22

Asymmetric Laplace Isomorphic Conic Encoding — Technical Documentation

Analytic Inverse Transform Sampling	22
Dynamic Multiple Importance Sampling	23
References	23

Open Source License

Document body (text, figures and theoretical exposition): licensed under **Creative Commons Attribution 4.0 International (CC BY 4.0)**

(<https://creativecommons.org/licenses/by/4.0/>).

Anyone is free to copy, modify, redistribute and use this document commercially, provided the author is attributed; when redistributing or adapting, retain this attribution and license notice and indicate if the original document was modified.

Accompanying code (GLSL / HLSL shader implementation): remains under the **MIT License**, free of charge for academic research, commercial game engine development and offline renderer development; retain the original author attribution and this notice when distributing or using the core algorithm code.

Attribution: Author sjrsjz (GitHub: sjrsjz / sjrsjz@gmail.com), title “Asymmetric Laplace Isomorphic Conic Encoding — Technical Document” .

Preface

Background

Diffuse lighting reconstruction is an important problem in computer graphics. Its goal is to reconstruct a lighting vector from a set of lighting samples for use in subsequent rendering. In practical applications, due to noise in the lighting samples, the reconstructed lighting vector is often contaminated by noise, necessitating a denoising step.

Note: Essentially, I found SVGF [1] too poor and difficult to tune with its unacceptably blurry cold-start, SH too heavy and prone to negative values and ringing artifacts, SG prohibitively slow, ReSTIR [2] not a true denoising pass, and NRD [3] both beyond my grasp and ill-suited for MC shaders (being excessively heavyweight). This directly forced me to produce the work presented below.

Purpose

This document aims to introduce a novel lighting encoding scheme along with its corresponding denoising solution and reconstruction algorithm.

Modeling

First Principles

We define the lighting state space directly as the augmented cone:

$$\mathcal{C} = \{(\mathbf{v}, \omega) \in \mathbb{R}^n \times \mathbb{R}_{\geq 0} \mid \omega \geq |\mathbf{v}|\}$$

An arbitrary lighting encoding is represented as a 2-tuple $L = (\mathbf{v}, \omega) \in \mathcal{C}$, where $\mathbf{v} \in \mathbb{R}^n$ is the directional moment and $\omega \in \mathbb{R}_{\geq 0}$ is the total incident radiance (sum of per-sample scalar magnitudes). The lighting synthesis operator $T : \text{List}(\mathcal{C}) \rightarrow \mathcal{C}$ maps a set of signal samples to a synthesized lighting state, where $\mathcal{L} \in \text{List}(\mathbb{R}^n)$ is the input list of Monte Carlo samples.

Each Monte Carlo sample is a probe ray: its BRDF modulation is deferred to the shading stage, so the raw signal $x_i = \hat{\mathbf{d}}_i \cdot \mathbf{L}_i$ encodes the material-independent

incident radiance L_i arriving from direction $\hat{d}_i$. This probe-based architecture ensures the ALICE encoding captures the full incident light field regardless of surface material (diffuse, metallic, or translucent).

Design rationale: the choice of state space $\mathcal{C}$ is not an arbitrary stipulation, but a natural consequence jointly determined by the signal structure and probability theory (see the “Probabilistic Origin of the Cone Constraint” section for details).

This modeling proceeds entirely from algebraic first principles and a signal-processing perspective (excluding additional physical assumptions) to derive the analytic form of the lighting synthesis operator T .

The operator satisfies the following algebraic axioms:

1. Axiom 1 — Associativity & Commutativity (signal order-independence)

The synthesis result is independent of the input order of samples and computational grouping. For any sample list $\mathcal{L}$ and any disjoint partition $\mathcal{L} = \bigcup_k \mathcal{L}_k$, the operator satisfies:

$$T(\mathcal{L}) = T(\{T(\mathcal{L}_k)\})$$

Signal perspective: Monte Carlo samples form an unordered data stream; statistical extracted quantities should not depend on sample arrival order. Mathematically, this axiom provides Lie group symmetry guarantees for the underlying topology of the accumulation operation.

2. Axiom 2 — Radiance Conservation (L1 amplitude additivity)

The total radiance of the system is strictly conserved before and after synthesis. Radiance is defined as the L1 norm of the signal. For any sample list $\mathcal{L} = \{x_i\}$:

$$\omega(T(\mathcal{L})) = \sum_{x_i \in \mathcal{L}} |x_i|$$

Signal perspective: this is L1-norm additivity — the most fundamental amplitude conservation law in signal processing. In the Monte Carlo context, the total radiance of the synthesized lighting equals the sum of radiance values from all sampled rays.

3. Axiom 3 — Directional Moment Fidelity (first-moment additivity)

The combined directional moment equals the vector superposition of the directional moments of each sample. For any sample list $\mathcal{L} = \{x_i\}$:

$$v(T(\mathcal{L})) = \sum_{x_i \in \mathcal{L}} x_i$$

Equivalent formulation: for any linear probe direction $a \in \mathbb{R}^n$, its linear response to the synthesized lighting equals the superposition of the responses to all individual input samples:

$$a \cdot v(T(\mathcal{L})) = \sum_{x_i \in \mathcal{L}} a \cdot x_i$$

Signal perspective: this is the additivity of the signal first moment (mean). The directional moment v is the vector mean of the signal in $\mathbb{R}^n$ multiplied by the number of samples; its linear additivity is a fundamental structural property of signal space.

4. Axiom 4 — Positive Homogeneity

When all inputs are scaled uniformly, the synthesized result is scaled by the same factor. For any non-negative scalar $\lambda \geq 0$:

$$T(\{\lambda \mathbf{x}_i\}) = \lambda T(\{\mathbf{x}_i\})$$

Signal perspective: when the signal is uniformly amplified by factor λ , all statistical extracted quantities scale proportionally — the scale-covariance axiom in signal processing.

We define the auxiliary mean operator

$$T_{\text{avg}}(\mathcal{L}) = \frac{1}{|\mathcal{L}|} T(\mathcal{L})$$

to represent the average lighting synthesis result of the input samples.

Operator Derivation

Direct Derivation

From Axiom 2 (Radiance Conservation) and Axiom 3 (Directional Moment Fidelity), the analytic form of the synthesis operator T is **directly and uniquely determined**:

$$T(\mathcal{L}) = \left(\sum_{\mathbf{x}_i \in \mathcal{L}} \mathbf{x}_i, \sum_{\mathbf{x}_i \in \mathcal{L}} |\mathbf{x}_i| \right) \in \mathcal{C}$$

Its corresponding mean operator is:

$$T_{\text{avg}}(\mathcal{L}) = (E[\mathbf{x}], E[|\mathbf{x}|])$$

Cone Closure Proof

We must verify $T(\mathcal{L}) \in \mathcal{C}$, i.e., prove $\omega \geq |v|$:

$$\omega = \sum_i |\mathbf{x}_i| \geq \left| \sum_i \mathbf{x}_i \right| = |v|$$

This follows directly from the triangle inequality. $\square$

Linearity

Introducing the augmented signal representation $\tilde{\mathbf{x}} = (\mathbf{x}, |\mathbf{x}|) \in \mathbb{R}^{n+1}$, the synthesis operator reduces to pure vector addition:

$$T(\mathcal{L}) = \sum_{\mathbf{x}_i \in \mathcal{L}} \tilde{\mathbf{x}}_i$$

The mean operator is simply the mathematical expectation of the augmented signal:

$$T_{\text{avg}}(\mathcal{L}) = E[\tilde{\mathbf{x}}]$$

Uniqueness Proof

The uniqueness of the synthesis operator is **independently and completely** guaranteed by Axiom 2 and Axiom 3:

- **Axiom 2** independently and completely locks the scalar component: $\omega(T(\mathcal{L})) = \sum |\mathbf{x}_i|$ is the unique scalar assignment satisfying Radiance Conservation.

- **Axiom 3** independently and completely locks the vector component: $v(T(\mathcal{L})) = \sum x_i$ is the unique vector assignment satisfying Directional Moment Fidelity.
- Axiom 1 (Associativity & Commutativity) provides Lie group symmetry guarantees for the accumulation operation, ruling out any non-commutative or non-associative synthesis schemes.
- Axiom 4 (Positive Homogeneity) rules out any nonlinear dependence on sample count or magnitude.

The two core axioms (2 and 3) each independently and uniquely determine one component of the output state; no other operator can simultaneously satisfy both axioms. Hence the analytic form of T is unique. $\square$

Probabilistic Origin of the Cone Constraint

This section explains why the cone constraint $\omega \geq |v|$ on state space $\mathcal{C}$ is not an arbitrary stipulation but an inevitable consequence of probability theory.

Jensen's Inequality Guarantee

For any probability measure μ defined on $\mathbb{R}^n$ (with finite first moment and finite first absolute moment), by Jensen's inequality:

$$E_{\mu}[|x|] \geq |E_{\mu}[x]|$$

i.e., $\omega \geq |v|$. Therefore, any lighting state parameterized by $(v, \omega) = (E[x], E[|x|])$ **automatically falls within the cone** $\mathcal{C}$.

Reinterpretation of Boundary States

The cone boundary $\omega = |v|$ corresponds to Jensen's inequality taking equality, which occurs if and only if the distribution degenerates to a Dirac δ function — i.e., all signal samples point in the same direction. This precisely corresponds to the raw single-sample ray state in the Monte Carlo pipeline before any spatial filtering:

$$\text{single-sample: } (v, \omega) = (x_i, |x_i|), \quad \omega = |v| \quad (\text{cone boundary})$$

After spatial filtering and accumulation of multiple samples, due to the dispersion of the empirical distribution, Jensen's inequality holds strictly:

$$\text{multi-sample: } \omega = E[|x|] > |E[x]| = |v| \quad (\text{cone interior})$$

Statistical Meaning of the Jensen Gap

Define the Jensen gap:

$$I = \omega - |v| = E[|x|] - |E[x]| \geq 0$$

I measures how far the signal distribution is from a Dirac state (perfectly directional). In the current implementation it is the linear cone coordinate that enforces $\omega \geq |v|$: $I = 0$ denotes a pure directional state on the cone boundary, while $I > 0$ denotes directional spread.

Algorithmic Significance

This section elaborates the engineering value of the above algebraic structure for real-time rendering pipelines.

In computer graphics and real-time rendering, spatial filtering and denoising algorithms (such as SVGF and its derivative architectures) inherently rely heavily on various linear or convex combination operations (weighted summation, convolution, etc.).

The mathematical derivation above provides a powerful proof: **the synthesis operator T is pure vector addition in the augmented signal space**. In a real-world pipeline, we only need to perform extremely low-cost ordinary linear blending of samples in the augmented representation $\tilde{x} = (x, |x|)$ (computing T_{avg}), and the final filtered result will rigorously conform to all algebraic axioms and the Radiance Conservation law.

Because T is inherently linear, **no post-hoc nonlinear correction is needed** — every operation in the filtering loop is a strict linear combination, and irradiance reconstruction $E(\hat{n})$ is deferred to a single pass at the shading output stage. This supports a lighting-denoising pipeline that combines linear moment preservation with efficient shading.

Statistical Model

Runtime Closure

The ALICE buffer state remains the linear first moment

$$\vec{L} = (v, \omega), \quad |v| \leq \omega$$

, where ω is total incident energy and v is the directional first moment. Blending, temporal accumulation, and spatial filtering all operate directly in this four-dimensional cone.

The runtime closure uses the radial reference measure $d\nu = r dr d\Omega$. Let $\kappa = \rho = |v|_{\omega}^{\perp}$ and $\hat{v} = \frac{v}{|v|}$. The joint density is

$$p_{L(r, \vec{u})} = \frac{\beta^2(1 - \kappa^2)}{4\pi} \exp(-\beta r(1 - \kappa \hat{v} \cdot \vec{u})), \quad \beta = \frac{2}{\omega(1 - \kappa^2)}.$$

After integrating out the radius, the normalized spherical energy density is

$$p_{E(\vec{u})} = \frac{(1 - \kappa^2)^2}{4\pi(1 - \kappa \hat{v} \cdot \vec{u})^3}.$$

It satisfies $\int_{S^2} p_E d\Omega = 1$ and $\int_{S^2} \vec{u} p_E d\Omega = \kappa \hat{v}$ exactly. Therefore κ is the normalized first-moment length and requires no numerical inversion. The $\kappa = 0$ state is uniform on the sphere; as $\kappa \rightarrow 1$, the closure converges to a directional atom on its axis.

Second-Order Statistics and Dual Parameters

Along and perpendicular to the principal axis, the single-sample spatial covariance components are

$$\sigma_{\text{perp}}^2 = (1 - \kappa^2) \frac{\omega^2}{2}, \quad \sigma_{\text{parallel}}^2 = (1 + \kappa^2) \frac{\omega^2}{2}.$$

The scalar and radial variances used by the implementation are

$$\text{Var}_{\text{scalar}} = (3 - \kappa^2) \frac{\omega^2}{2} = 3 \frac{\omega^2}{2} - |v|_2^2, \text{Var}(R) = (1 + \kappa^2) \frac{\omega^2}{2}.$$

If the buffer stores $\frac{\text{Var}_{\text{scalar}}}{N_{\text{eff}}}$, the radial estimator variance is recovered as

$$\frac{\text{Var}(R)}{N_{\text{eff}}} = \frac{\text{Var}_{\text{scalar}}}{N_{\text{eff}}} \cdot \frac{1 + \kappa^2}{3 - \kappa^2}.$$

The natural parameters used by the divergence metric are

$$\theta = \beta \kappa \hat{v} = \beta \frac{v}{\omega}, \quad \beta = \frac{2}{\omega(1 - \kappa^2)}.$$

Writing $\varphi = (\theta, -\beta)$ and $\psi = (v, \omega)$ gives $\varphi \cdot \psi = -2$. The symmetric Jeffreys divergence between two states is evaluated as

$$D_J = \beta_1 \omega_2 + \beta_2 \omega_1 - \theta_1 \cdot v_2 - \theta_2 \cdot v_1 - 4.$$

Spatiotemporal filtering weights it by $W_{\text{eff}} = N_1 \frac{N_2}{N_1 + N_2}$.

Lambert Query

Let $\mu = \hat{v} \cdot \vec{n}$ and $d = \sqrt{1 - \kappa^2 + \kappa^2 \mu^2}$. The normalized clamped-cosine response is analytic:

$$e(\kappa, \mu) = \frac{1 - \kappa^2 + 2\kappa^2 \mu^2}{4d} + \kappa \frac{\mu}{2}, \quad E = \omega e.$$

To avoid cancellation for back-facing, highly concentrated states, the implementation uses the equivalent branch

$$e(\kappa, \mu) = \frac{(1 - \kappa^2)^2}{4d(d - \kappa\mu)^2}$$

when $\kappa\mu < 0$. Its boundary behavior is $e(0, \mu) = \frac{1}{4}$ and $\lim_{\kappa \rightarrow 1} e(\kappa, \mu) = \max(\mu, 0)$.

EON Query

The rough-diffuse path consumes the same (v, ω) state. The implementation separates the EON response into the analytic Lambert term, the Fujii–Oren–Nayar directional partition, and the multiple-scattering compensation. An Iris custom 3D RGBA16F LUT reconstructs the directional partition, with its four channels holding piecewise cubic Bernstein controls over transformed κ_i ; stable closed-form branches evaluate the missing-energy term without a second texture access. Uniform and directional-atom states use exact boundary paths.

GLSL Interface

The core implementation lives in `shaders/lib/lighting/maxent.glsl` and `shaders/lib/lighting/eon.glsl`. Callers continue to pass `vec4(v, omega)`; the buffer layout is unchanged:

```
float kappa = clamp(length(v) / omega, 0.0, 1.0 - 1e-6);
float response = maxent_irradiance(vec4(v, omega), normal);
```

```
vec3 outgoing = eon_project_maxent(  
    maxEntY, CoCg, normal, wo, roughness, albedo);
```

Denoising Pipeline Implementation

Pipeline Overview

The ALICE denoising pipeline adopts a multi-pass architecture, performing progressive filtering of the diffuse lighting signal in the spatiotemporal domain. The overall pipeline consists of the following core passes (see Dirt RT/shaders/post):

Pass	Shader	Function
100	fragment	Temporal accumulation: reprojects history frames using motion vectors, performs temporal recursive filtering in ALICE augmented space
swap2	compute	Variance pre-filtering: 5×5 geometry-aware bilateral filter smooths raw ALICE variance; 3σ energy clamping
300_cs	compute	Spatial filtering L1—L3: à-trous wavelet decomposition $R_0 \in \{1, 2, 4\}$, accelerated via shared memory
300	fragment	Spatial filtering L4—L6: à-trous wavelet decomposition $R_0 \in \{8, 16, 32\}$, rotation jitter decorrelation
swap3	compute	Buffer swap: writes filtered results back to the main buffer, completes double-buffer flip, saves historical statistics

Table 1: ALICE Denoising Pipeline Data Flow

Pass 100 (Temporal Accumulation) uses motion vectors to reproject the history frame’ s ALICE encoding to the current frame, performing exponential moving average in the augmented space. The details of this pass are beyond the scope of this section, which focuses on the three core passes of spatial-domain denoising: swap2 (variance pre-filtering), 300/300_cs (spatial filtering), and swap3 (buffer swap).

⚠ Critical Note: Although this denoiser references the à-trous wavelet decomposition framework of SVGF [1] in its architecture, it differs fundamentally from standard SVGF in key dimensions including **signal domain, weight functions, variance estimation, and preprocessing strategy**. The following sections explicitly mark these differences at the relevant positions. A summary of the core differences is provided in the table below.

Dimension	Standard SVGF	This Denoiser
Signal Domain	RGB 3-channel color (radiometric space)	ALICE augmented representation (v, ω) (4D linear space), chrominance (C_o, C_g) filtered independently
Synthesis Operator	Nonlinear (must handle nonlinear combinations in color space)	Purely linear T operator — sample accumulation in augmented space is simply vector addition, guaranteed by the ALICE algebraic structure
Variance Estimation	Local empirical variance over color channels	Closed-form theoretical variance $\text{Var}_{\text{scalar}(\mathbf{X})}$ of the ALICE maximum entropy distribution (depends only on first-order moments (v, ω)), with a dedicated 5×5 pre-filter pass
Luminance Weight	RGB luminance gradient + variance normalization	ALICE vector space distance $ v_{\text{center}} - v_{\text{sample}} $ + pre-filtered variance normalization
Energy Clamping	None	3σ energy clamping (swap2), preserving $\rho = \frac{ v }{\omega}$ unchanged
Temporal Accumulation	Exponential moving average in RGB color space	Direct linear accumulation in ALICE augmented space, w is exactly the effective frame count N_{eff}
Output Signal	Filtered RGB color	Filtered $(v, \omega) + (C_o, C_g)$, irradiance reconstruction $(E(\vec{n}))$ deferred to a single shading stage

Table 2: Key Differences between the ALICE Denoiser and Standard SVGF

Engineering Characteristics of the ALICE Denoiser:

Within the **diffuse channel**, the ALICE denoiser achieves excellent joint denoising results without needing to separate Direct Illumination (DI) from Global Illumination (GI) — the ALICE probe encodes a material-independent incident light field, where DI (sun contribution) and GI (indirect bounces) accumulate naturally in the augmented space. Specular reflection and refraction channels use independent denoising pipelines

(reflectIlluminationBuffer / refractIlluminationBuffer), architecturally decoupled from ALICE.

For purely diffuse or opaque dielectric materials, the diffuse channel handles the vast majority of the denoising load; for pure metals and other materials with negligible diffuse response, the specular reflection channel operates independently.

Actual performance: **contact shadows converge to clearly recognizable quality within 5—10 frames** (approximately 80—170 ms at 60 fps), making the convergence process virtually imperceptible to human vision.

Unified Diffuse Buffer

The ALICE denoising pipeline uses a unified SSBO (Shader Storage Buffer Object) on the host side to manage diffuse data, replacing the traditional multi-texture approach. The per-pixel data structure `UnifiedDiffuseElement` contains 18 `floats` (72 bytes), covering the following functional domains:

1. **RT Output Domain (12B)**: The raw ALICE encoding of the current frame's ray-traced output (`rt_aliceY_xy`, `rt_aliceY_zw`, `rt_CoCg`), written by `ray0.rgen`, read by Pass 100.
2. **Current Geometry Domain (20B)**: World-space position (p_x, p_y, p_z) + octahedron-compressed normal `oct_n` + secondary normal `oct_n2`.
3. **History Geometry Domain (16B)**: The previous frame's world-space position and compressed normal, used for edge-stop decisions during temporal reprojection. This domain is independent of the current geometry domain because `ray0.rgen` overwrites the current geometry domain every frame while the history geometry must persist across frames.
4. **Temporal History Domain (14B)**: The previous frame's accumulated ALICE encoding and accumulation weight, written by `swap3`, read by Pass 100.
5. **Swap Buffer Domain (14B)**: The current frame's to-be-filtered / already-filtered ALICE encoding and weight, serving as the data bus between Pass 100 → `swap2` → 300 → `swap3`.

ALICE encodings are stored using half-precision floating-point (`float16`) compression: each lighting state (v, ω) as a `vec4` is packed into two `floats` (via `packHalf2x16` / `unpackHalf2x16`), and the chrominance component `CoCg` is packed into one `float`, making a total of 3 `floats` to fully represent a single ALICE lighting state. The compression/decompression interface is as follows:

```
vec3 packAlice(AliceEncoding encoded) {
    float s0 = uintBitsToFloat(packHalf2x16(vec2(encoded.aliceY.x,
    encoded.aliceY.y)));
    float s1 = uintBitsToFloat(packHalf2x16(vec2(encoded.aliceY.z,
    encoded.aliceY.w)));
    float s2 = uintBitsToFloat(packHalf2x16(vec2(encoded.CoCg.x,
    encoded.CoCg.y)));
    return vec3(s0, s1, s2);
}
```

```

AliceEncoding unpackAlice(float s0, float s1, float s2) {
    AliceEncoding encoded;
    vec2 v0 = unpackHalf2x16(floatBitsToUint(s0));
    vec2 v1 = unpackHalf2x16(floatBitsToUint(s1));
    vec2 v2 = unpackHalf2x16(floatBitsToUint(s2));
    encoded.aliceY = vec4(v0.x, v0.y, v1.x, v1.y);
    encoded.CoCg = v2;
    return encoded;
}

```

Variance Pre-Filtering (swap2)

Design Motivation

The spatial filter of this denoiser critically depends on per-pixel variance estimates σ^2 . However, the raw variance produced by single-frame Monte Carlo sampling contains extremely high-frequency noise; using it directly to guide à-trous filtering would cause: (1) the luminance weight $w_{\text{luma}} = \Delta \frac{E}{\sigma}$ to become unstable at noisy pixels, producing blocky artifacts; (2) low-variance regions being misidentified as high-confidence, leading to edge blurring.

Difference from standard SVGF: Standard SVGF uses local empirical variance over color channels, recomputed at each à-trous level as $\sigma^2 = \max(0, \frac{1}{n} \sum x_i^2 - \mu^2)$. This denoiser instead uses the closed-form theoretical variance $\text{Var}_{\text{scalar}}(\mathbf{X})$ of the ALICE maximum entropy distribution, which depends only on the first-order moments (v, ω) and the effective frame count N_{eff} , requiring no storage of second-order moments. More importantly, we introduce a dedicated **variance pre-filter pass** (swap2) that performs 5×5 geometry-aware bilateral smoothing on the raw variance before entering the à-trous iteration. This preprocessing step is one of the key design elements distinguishing this denoiser from standard SVGF.

Therefore, before entering the à-trous iteration, a mild pre-smoothing of the raw ALICE variance is required. swap2 performs 5×5 geometry-aware bilateral filtering on 16×16 workgroups, simultaneously completing 3σ energy clamping and data push to the colortex outputs.

Cooperative Shared-Memory Loading

swap2 uses a 20×20 shared memory tile (16×16 workgroup + 2px halo), with 256 threads sharing the loading of 400 pixels via Round-Robin scheduling. Each tile element contains position, normal, raw variance, and total energy ω .

The raw variance is computed on-the-fly by `alice_estimator_variance(aliceY, weight)`, where `weight` is the temporally accumulated effective frame count N_{eff} . The variance formula uses the 3D scalar variance closed-form solution derived earlier (see the section “Variance Structure in the Original Sample Space $\mathbb{R}^n$ ”):

$$\text{Var}_{\text{scalar}}(\mathbf{X}) = \frac{2\omega^2 + \omega\sqrt{4\omega^2 - 3|v|^2}}{3} - \frac{1}{2}|v|^2$$

$$\sigma_{\text{raw}}^2 = \frac{\text{Var}_{\text{scalar}}(\mathbf{X})}{\max(N_{\text{eff}}, 1)}$$

3-Sigma Energy Clamping

Before performing spatial filtering on the ALICE encoding, swap2 first applies 3σ clamping to the output energy ω of the center pixel. In a 5×5 neighborhood, the weighted mean μ_ω and standard deviation σ_ω are computed (weights $w_{\text{kernel}} \cdot w_{\text{geom}}$), and the center pixel's ω is clamped to $[\mu_\omega - 3\sigma_\omega, \mu_\omega + 3\sigma_\omega]$:

$$\text{scale} = \frac{\omega_{\text{clamped}}}{\max(\omega_{\text{center}}, 10^{-8})}, \quad \mathbf{v}' = \mathbf{v} \cdot \text{scale}, \quad (\text{Co}, \text{Cg})' = (\text{Co}, \text{Cg}) \cdot \text{scale}$$

After clamping, the entire ALICE vector is proportionally scaled to preserve the anisotropy $\rho = \frac{|\mathbf{v}|}{\omega}$. This operation preserves the cone constraint $\omega \geq |\mathbf{v}|$ while preventing extreme energy outliers caused by insufficient temporal accumulation from propagating through the subsequent à-trous filtering.

5×5 Geometry-Aware Variance Filtering

The variance filter uses a B-spline kernel $h(x) \in \{1.0, 0.66667, 0.44444\}$ with 5×5 separable weights:

$$w_{\text{kernel}}(k_x, k_y) = h(|k_x|) \cdot h(|k_y|)$$

The geometric weights adopt the standard SVGF formula:

$$w_{\text{normal}} = (\text{clamp}(\mathbf{n}_{\text{center}} \cdot \mathbf{n}_{\text{sample}}, 0, 1))^{\gamma_n}$$

$$w_{\text{depth}} = \exp\left(-\frac{|(\mathbf{p}_{\text{sample}} - \mathbf{p}_{\text{center}}) \cdot \mathbf{n}_{\text{center}}|}{\sigma_p \cdot \text{footprint}}\right)$$

The total weight is the product of three factors $w = w_{\text{kernel}} \cdot w_{\text{normal}} \cdot w_{\text{depth}}$, and the filtered variance is the weighted average (with an optional conservative mode using $\max(\text{filtered}, \text{center})$ to guarantee variance is never underestimated). If computational cost is a concern, the normal weight can be approximated using an $\exp$ form.

Gaussian Curvature Edge Marking (Optional, Non-Essential)

⚠️ Optional Feature: Gaussian curvature edge marking is an **experimental auxiliary feature** of this denoiser, controlled by the compile-time macro `ENABLE_GAUSSIAN_FILTER`. In most scenarios, the basic geometric weights (normal + depth) are already sufficient to provide reliable edge stopping. This feature is not a necessary component of the denoiser, and disabling it does not affect core denoising quality. This procedure is only applicable to fragmented sub-pixel geometry that TAA cannot correctly handle, and its use should be avoided in practice.

At geometric edges (such as block corners, discontinuity boundaries), normal- and depth-based edge stopping cannot reliably separate lighting signals on either side — because the geometry buffer itself is already fractured at the edge. swap2 uses 3×3 finite differences to compute the local surface Gaussian curvature K :

$$E = (D_x \mathbf{p})^2, \quad F = (D_x \mathbf{p}) \cdot (D_y \mathbf{p}), \quad G = (D_y \mathbf{p})^2$$

$$L = D_{xx} \mathbf{p} \cdot \mathbf{n}, \quad M = D_{xy} \mathbf{p} \cdot \mathbf{n}, \quad N = D_{yy} \mathbf{p} \cdot \mathbf{n}$$

$$K = \frac{LN - M^2}{EG - F^2}$$

The first-order / second-order partial derivatives are estimated via central differences (D_x using ± 1 half-neighbor difference, D_{xx} using the second-order central difference of the three points $+1, 0, -1$). When $|K| > \tau$ (curvature threshold, defined by the `CURVATURE_THRESHOLD` macro), the pixel's ω is marked as negative. In subsequent `300/300_cs` passes, detecting $\omega < 0$ automatically zeroes the geometric weight (`geomValid = 0`), relying solely on the luminance weight for denoising and avoiding incorrect blending of signals across geometric discontinuities.

Spatial Filter (300 / 300_cs)

Difference from standard SVGF: This denoiser only references SVGF in the **iterative architecture** of the à-trous wavelet decomposition. The core of the weight functions — signal representation, distance metric, variance source — is entirely redesigned around the ALICE encoding and is fundamentally different from standard SVGF. Details follow.

À-Trous Wavelet Decomposition Architecture

The spatial filter uses a 6-level à-trous wavelet iteration (iterative architecture referencing [1]), with each level using a 3×3 filter kernel and the step size R_0 doubling at each level:

Level (STEP)	Step Size R_0	Shader	Effective Radius
1	1	300_cs (Compute)	1 px
2	2	300_cs (Compute)	3 px
3	4	300_cs (Compute)	7 px
4	8	300 (Fragment)	15 px
5	16	300 (Fragment)	31 px
6	32	300 (Fragment)	63 px

Table 3: À-Trous Iteration Level Configuration

The first 3 levels ($R_0 \leq 4$) execute in a compute shader (300_cs), leveraging shared memory (LDS) to reduce redundant texture reads; the last 3 levels ($R_0 \geq 8$) fall back to a fragment shader (300), because at large step sizes the neighborhood tile overlap rate is low (texture reads across threads no longer overlap significantly) and the LDS benefit diminishes.

Compute Shader Shared-Memory Optimization

The core optimization of `300_cs` lies in exploiting data reuse within a workgroup. For a 16×16 workgroup and step size R_0 , the required texture tile size is $(16 + 2R_0)^2$:

R_0	Tile Size	Geometry (vec4)	Lighting (vec4)
1	18×18	324	324
2	20×20	400	400
4	24×24	576	576

Table 4: Tile Sizes and Shared Memory Usage for Each R_0 Level

The shared memory usage for all tile sizes (maximum $576 \times 2 \times 16 = 18.0$ KB) is well below the typical GPU LDS limit of 32—64 KB. 256 threads share the loading task via Round-Robin scheduling (each thread loads $\lceil \frac{\text{TILE_AREA}}{256} \rceil$ elements), with out-of-bounds pixels having their variance written as negative values as a sky mask. After loading, synchronization via `barrier() + memoryBarrierShared()` ensures that all subsequent 3×3 à-trous sampling loops read entirely from shared memory, reducing the global texture read count by approximately 3—6×.

Tri-Lateral Weight Function

The composite weight of each neighborhood sample is formed by the product of three factors.

Kernel Weight: A B-spline kernel $h \in \{1.0, 0.66667\}$, distinguishing only the center and cross-shaped neighbors:

$$w_{\text{kernel}(i,j)} = h(|i|) \cdot h(|j|)$$

Geometry Weight:

$$w_{\text{geom}} = \gamma_n (1 - \mathbf{n}_{\text{center}} \cdot \mathbf{n}_{\text{sample}}) + \frac{|(\mathbf{p}_{\text{sample}} - \mathbf{p}_{\text{center}}) \cdot \mathbf{n}_{\text{center}}|}{\sigma_p \cdot \text{footprint}} \cdot \text{geomValid}$$

where:

- γ_n is the normal sensitivity parameter (SVGF_NORMAL_POWER), controlling the penalty strength for normal differences;
- σ_p is the position sensitivity parameter (SVGF_POSITION_PARAM);
- $\text{footprint} = \max\left(\frac{\text{distToCam}}{\text{resolution.y}}, 10^{-4}\right)$ is the world-space footprint size of the pixel, making the depth term independent of screen resolution;
- `geomValid` is controlled by the Gaussian curvature flag: when the curvature exceeds the threshold, this term is zeroed, downgrading the filter to purely luminance-driven.

Luminance Weight:

Difference from standard SVGF: Standard SVGF' s luminance weight is based on the RGB color space gradient $|L_i - L_j|$ (where L_i is the pixel luminance). This denoiser measures differences in the ALICE augmented space — using the Euclidean distance of the direction vector $|\mathbf{v}_{\text{center}} - \mathbf{v}_{\text{sample}}|$. The mathematical justification for this choice is: $\mathbf{v}$ is precisely the linearly additive signal component in the augmented space $\mathcal{C}$, and its Euclidean distance directly measures the difference in the joint direction-energy space of the lighting, without going through an irradiance reconstruction step. Furthermore, since the variance pre-filter (swap2) has already smoothed σ^2 , the center pixel' s pre-

filtered variance is used directly as the normalization baseline (standard SVGF uses locally computed $\sigma_{\text{center}}^2 + \sigma_{\text{sample}}^2$ at each à-trous level):

$$w_{\text{luma}} = \frac{|v_{\text{center}} - v_{\text{sample}}|}{\sqrt{\sigma_{\text{center}}^2}} \cdot \varphi_l$$

where φ_l is the luminance sensitivity parameter (SVGΦ_PHI_L).

Critical Note on the Asymmetric σ^2 Choice

The use of the asymmetric form σ_{center}^2 rather than the symmetric form $\sigma_{\text{center}}^2 + \sigma_{\text{sample}}^2$ is a core design decision of this denoiser, verified through extensive experimentation. The logic chain behind it critically depends on the existence of the variance pre-filter (swap2):

Case 1: Without Variance Pre-Filtering (Standard SVGF Scheme)

If the raw, un-pre-smoothed variance is used directly in the à-trous filter, the symmetric form $\max(\sigma_{\text{center}}^2 + \sigma_{\text{sample}}^2, 10^{-8})$ must be adopted. The reason: the raw variance signal is extremely noisy, and single-pixel variance estimates are highly unstable. If only σ_{center}^2 is taken, the center pixel's variance may oscillate violently between noise peaks and troughs — when σ_{center}^2 happens to be extremely small (a variance trough) while $|v_{\text{center}} - v_{\text{sample}}|$ is large due to noise, the luminance weight w_{luma} inflates abnormally, causing neighborhood samples to be over-trusted, resulting in **black speckle artifacts and severe energy loss** (denoiser collapse). The introduction of σ_{sample}^2 is essentially a form of statistical smoothing — the superposition of two independent noise terms reduces the weight's sensitivity to single-point variance anomalies.

Case 2: With Variance Pre-Filtering (This Denoiser's Scheme)

The 5×5 bilateral smoothing of swap2 has already substantially eliminated high-frequency noise from the variance signal, and σ_{center}^2 itself is already sufficiently stable. If the symmetric form $\sigma_{\text{center}}^2 + \sigma_{\text{sample}}^2$ continues to be used at this point, not only is the stabilizing effect of σ_{sample}^2 already negligible — more critically, σ_{sample}^2 and σ_{center}^2 , though pre-filtered, still retain residual minute differences spatially. These differences fluctuate across frames as the sampling pattern changes, and after two-sample superposition, they produce **extremely severe temporal flickering side effects** in w_{luma} . This flickering cannot be fully eliminated by temporal accumulation because the inter-frame fluctuation of the luminance weights directly alters the effective kernel shape at every à-trous level.

Conclusion: Variance pre-filtering and single-sample σ_{center}^2 are a tightly coupled design pair — the pre-filter eliminates the instability of σ_{center}^2 , thereby making it safe to remove the σ_{sample}^2 term, which in turn eradicates the temporal flickering caused by two-sample variance superposition. This is yet another critical and non-obvious point of divergence from standard SVGF.

Parameter Sensitivity of the Variance Floor

In the actual implementation, σ^2 is clamped to a lower bound ε before entering $\frac{1}{\sqrt{\sigma^2}}$:

$$\sigma^2 = \max(\sigma_{\text{center}}^2, \varepsilon), \quad \varepsilon = 10^{-9}$$

(In the symmetric scheme, this would be $\max(\sigma_{\text{center}}^2 + \sigma_{\text{sample}}^2, 10^{-8})$, with the same principle.)

ε is the most sensitive parameter in this denoiser governing the tradeoff between **contact shadow quality** and **stability in dark regions**. Its mechanism of action is as follows:

ε controls the upper bound of $\frac{1}{\sqrt{\sigma^2}}$ — since $\frac{1}{\sqrt{\sigma^2}} \leq \frac{1}{\sqrt{\varepsilon}}$. When σ_{center}^2 naturally tends to extremely small values in dark regions, $\frac{1}{\sqrt{\sigma^2}}$ would diverge to infinity without an upper bound, causing w_{luma} to completely dominate the composite weight and the denoiser to over-respond to any minute v differences.

- **ε too large (e.g., 10^{-6}):** The denoiser becomes “sluggish” — $\frac{1}{\sqrt{\sigma^2}}$ is suppressed, and w_{luma} in dark regions cannot distinguish genuine lighting boundaries from noise; **contact shadows in low-energy regions are blurred**.
- **ε too small (e.g., 10^{-12}):** The denoiser becomes “overly sensitive” — $\frac{1}{\sqrt{\sigma^2}}$ is extremely high in dark regions, and w_{luma} over-responds to noise fluctuations, manifesting as **denoising collapse** (structural black spots in dark regions) or **shadow creeping** (inter-frame noise amplified by weights forming slowly drifting artifacts).
- **$\varepsilon = 10^{-9}$:** The balance point selected after extensive experimentation across diverse scenes (indoor dark corners, dense forest shadows, caves), providing sufficient numerical stability for dark regions while preserving contact shadow sharpness.

Composite Weight: The luminance weight appears in a “bilateral enhancement” form at two positions in the product — as an exponential decay term penalizing large energy differences, and simultaneously as a prefactor providing mild adaptive enhancement:

$$w_0 = w_{\text{kernel}} \cdot (1 + w_{\text{luma}}) \cdot \exp(-(w_{\text{geom}} + w_{\text{luma}}))$$

Difference from standard SVGF: Standard SVGF’s composite weight is $w_{\text{kernel}} \cdot \exp(-(w_{\text{geom}} + w_{\text{luma}}))$, i.e., relying solely on exponential decay. The $(1 + w_{\text{luma}})$ prefactor introduced by this denoiser additionally provides first-order luminance-adaptive enhancement, delivering faster convergence and stronger denoising capability in high-variance regions (where w_{luma} is large). This modification is particular to the ALICE augmented space — because w_{luma} is based on the Euclidean distance of v rather than an RGB gradient, its numerical range and statistical properties differ from standard SVGF.

Sample Accumulation in ALICE Augmented Space

Fundamental difference from standard SVGF: This is the most fundamental point of divergence between this denoiser and standard SVGF. Standard SVGF accumulates weighted color values $\sum_i w_i c_i$ in RGB color space — linear combinations in color space do not correspond to physical lighting synthesis. This denoiser accumulates in the ALICE augmented space $\mathcal{C}$ — guaranteed by the ALICE first principles, the T operator in the augmented space is pure vector addition, so $\sum_i w_i \tilde{x}_i$ is mathematically

strictly equivalent to the physical synthesis of lighting. The denoiser never needs to handle nonlinear synthesis logic at any step. This is the core advantage of ALICE encoding over traditional RGB denoising.

Neighborhood samples are synthesized through pure linear accumulation in the ALICE augmented space (corresponding to the linearity of the T operator derived earlier):

$$\mathbf{v}_{\text{accum}} = \sum_i w_i \mathbf{v}_i, \quad \omega_{\text{accum}} = \sum_i w_i \omega_i, \quad (\text{Co}, \text{Cg})_{\text{accum}} = \sum_i w_i (\text{Co}, \text{Cg})_i$$

After normalization, $\mathbf{v}_{\text{out}} = \frac{\mathbf{v}_{\text{accum}}}{\sum_i w_i}$, and similarly ω and chrominance are each divided by the total weight. The variance is propagated to the next level via the variance propagation formula for independent-sample weighted means:

$$\sigma_{\text{out}}^2 = \frac{\sum_i w_i^2 \sigma_i^2}{\left(\sum_i w_i\right)^2}$$

Rotation Jitter

At $R_0 \geq 8$ (STEP ≥ 4) levels, the fixed axis-aligned sampling pattern can produce structured grid artifacts. To eliminate these artifacts, `atrous_denoise_diffuse.glsl` introduces random rotation (Rotation Jitter) at each level:

$$\mathbf{d}_{\text{rotated}} = R(\theta) \cdot \mathbf{d}_{\text{aligned}}, \quad \theta = 2\pi \cdot \text{rand}(\text{pix} + R_0)$$

The rotation matrix $R(\theta)$ is a standard 2D rotation, with the random seed jointly hashed from the pixel coordinates and the current level step size R_0 , ensuring: (1) rotation angles are independently distributed across different pixels; (2) rotation angles are mutually uncorrelated across different levels; (3) the rotation angle for the same pixel at the same level remains fixed across frames (to avoid temporal flickering). Sampling coordinates are rounded (`round`) to prevent floating-point truncation errors from causing out-of-bounds access.

Special Handling of the Final Pass

Difference from standard SVGF: Standard SVGF's final level outputs only the filtered result. This denoiser additionally outputs a large-radius blurred version to an independent texture (`colortex5`), used by `swap3` to accelerate the convergence of low-confidence pixels. This dual-output design does not exist in standard SVGF.

The final level (STEP = 6), in addition to outputting the regular filtered result (`colortex4`), also outputs a "large-radius post-blurred" result to `colortex5` (`out_light_sample_blurred`). This result is read by `swap3` for fast convergence of low-weight pixels. Additionally, the Gaussian curvature flag (the sign of ω , a non-essential feature) is only propagated forward in non-final passes; the final pass output forces the absolute value to ensure that subsequent irradiance reconstruction is not disturbed by the sign marker.

Buffer Swap (swap3)

swap3 is the bridge connecting the current frame to the next frame, responsible for the following key tasks:

Double-Buffer Flip: Copies `data_swap` (the current frame's filtered result) to `data` (which will serve as the next frame's history), for Pass 100 to read during temporal reprojection in the next frame. This operation closes the data flow loop in the temporal dimension.

Historical Statistics Preservation: $w_{\text{prev}} = w$, saving the current frame's effective accumulation frame count as the historical weight. This weight is used in the next frame's Pass 100 for: (1) computing the temporal blending factor α ; (2) evaluating the reliability of the reprojected sample.

Filtered Result Read-Back: Reads the post-blurred ALICE encoding output by the final pass from `colortex5`, writing it into `data_swap` as the starting point for the next frame's spatial filtering.

Low-Weight Blending Acceleration (inspired by NRD [3], non-standard SVGF procedure):

Difference from standard SVGF: Standard SVGF's temporal-spatial relationship is unidirectional (temporal accumulation $\rightarrow$ spatial filtering), and the temporal history is never modified after spatial filtering. This denoiser performs a reverse blend in swap3 — using the large-radius spatial filtering result to backfill low-confidence temporal history. This design borrows ideas from NRD (NVIDIA Real-time Denoisers) [3] and does not exist in standard SVGF.

When the historical accumulation is insufficient ($w < 1.0$), the current historical data is gently blended toward the spatial filtering result:

$$\text{data} = \text{mix}\left(\text{data}, \text{blurred_alice}, \text{clamp}\left(\frac{1}{\max(w, 1.0)}, 0, 1\right)\right)$$

This operation provides additional convergence acceleration during cold starts, disocclusions, or temporal accumulation breakdowns caused by rapid camera motion. In practice, the impact of this blending on denoising quality is mild, but it helps eliminate isolated flickering pixels. The parameter $\frac{1}{\max(w, 1.0)}$ ensures that the blending intensity is inversely proportional to confidence: the smaller w is, the more the result is biased toward trusting the spatial filtering result. Likewise, this operation is not essential and, when disabled, does not significantly affect final quality in most scenarios.

Data Flow Summary

Integrating the three passes above, the complete data flow of the ALICE denoising pipeline is as follows:

1. **Pass 100 (Temporal Accumulation):** Reads the historical ALICE encoding $\vec{L}_{\text{hist}}$ from the SSBO, performs weighted averaging with the current frame's RT output $\vec{L}_{\text{curr}}$ in the augmented space, and outputs $\vec{L}_{\text{temp}}$ along with the accumulation weight w .

2. **swap2 (Variance Pre-Filtering)**: Reads $\vec{L}_{\text{temp}}$ and w , computes the raw variance σ_{raw}^2 , performs 5×5 bilateral smoothing + 3σ clamping + Gaussian curvature marking, and pushes the data to colortex3 (geometry) and colortex4 (ALICE + variance).
3. **300_cs (Spatial Filtering L1—L3)**: Cooperatively loads from colortex3/4 into LDS, performs à-trous filtering with $R_0 \in \{1, 2, 4\}$, and writes the result back to colortex4.
4. **300 (Spatial Filtering L4—L6)**: Directly texelFetch from colortex3/4, performs à-trous filtering with $R_0 \in \{8, 16, 32\}$ (including rotation jitter), and additionally outputs the final level to colortex5.
5. **swap3 (Buffer Swap)**: Double-buffer flip, saves historical statistics, reads back the blurred result, and closes the data flow loop. The next frame restarts from step 1.

In compressed representation, the spatial filtering stage of the ALICE denoiser requires only two `vec4s` (32 bytes total) to fully represent all necessary geometric information (one `vec4`) and lighting information (one `vec4`).

Naming and Scope

The lighting encoding retains the name Asymmetric Laplace Isomorphic Conic Encoding (ALICE). “Conic” denotes the linear state space $\mathcal{C} = \{(v, \omega) \mid \omega \geq |v|\}$. “Isomorphic” denotes the reversible mapping between the source representation (v, I) and the embedded representation $(v, \omega = |v| + I)$.

The current runtime closure is a statistical model selected for directional-energy reconstruction. Its reference measure is not three-dimensional Cartesian Lebesgue measure, so this document no longer identifies it with a Maxwell–Jüttner photon-gas model. Here κ denotes only normalized first-moment length and directional concentration, while β is a closure scale parameter rather than a physical drift velocity or thermodynamic temperature.

Screen-Space Path Rebuilding Importance Sampling

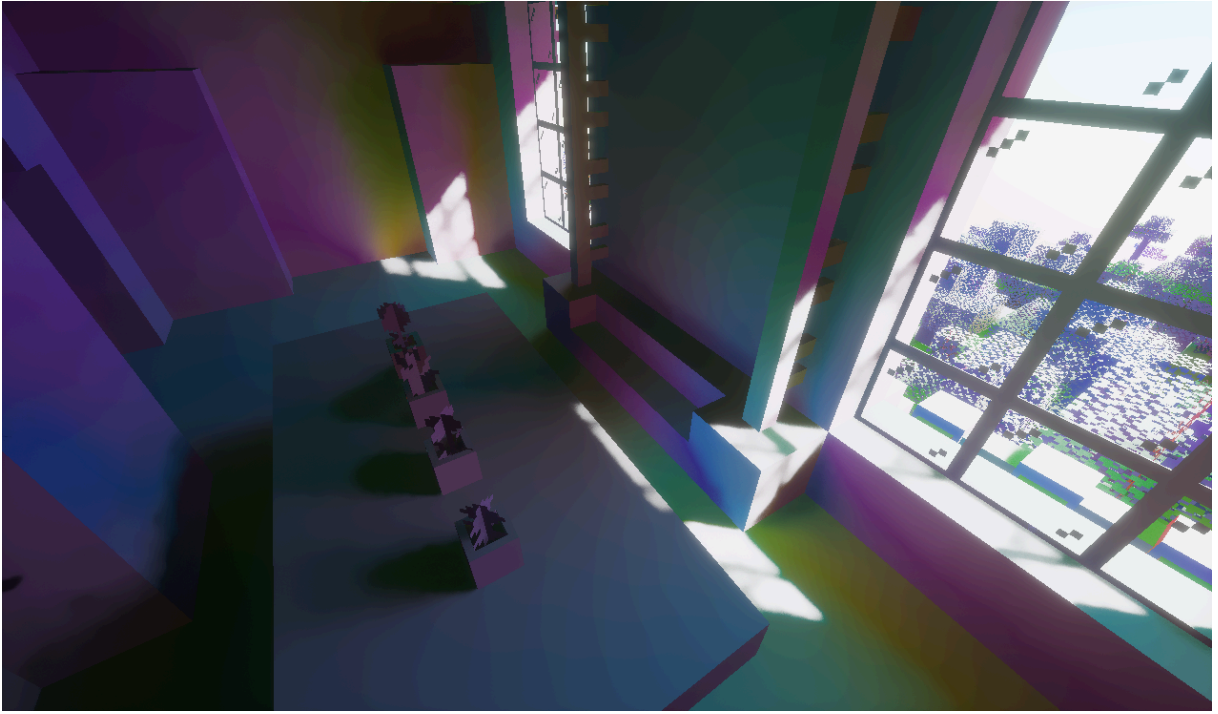


Figure 2: First-moment distribution of the light field obtained after denoising

Physical Motivation and Prior Distribution

In real-time path tracing, although Next Event Estimation (NEE) can effectively reduce the variance of direct lighting, for complex secondary bounces (such as deep corridors or room interiors with tiny windows), blind cosine-weighted sampling has extreme difficulty hitting effective light sources, causing indirect lighting to produce destructively high-frequency, long-tail noise (Fireflies).

Since we have already extracted and reconstructed the maximum entropy distribution state (v, ω) of the light field in the spatiotemporal domain using ALICE encoding within the denoising pipeline, we can naturally use it as **prior knowledge (Prior)** to perform Path Guiding over the hemispherical space when casting rays in the next frame.

The closure' s angular energy density is used directly as the guiding PDF:

$$p_{\text{ALICE}(\vec{u})} = \frac{(1 - \kappa^2)^2}{4\pi(1 - \kappa\hat{v} \cdot \vec{u})^3}.$$

This expression is already normalized over the full sphere S^2 , and its first moment is exactly $\kappa\hat{v}$.

Analytic Inverse Transform Sampling

Let $\mu = \hat{v} \cdot \vec{u}$. The marginal distribution obeys

$$(1 - \kappa\mu)^{-2} = \text{lerp}((1 + \kappa)^{-2}, (1 - \kappa)^{-2}, \xi_1).$$

For $\kappa > 0$, it can therefore be sampled directly with

$$\mu = \frac{1 - [\text{lerp}((1 + \kappa)^{-2}, (1 - \kappa)^{-2}, \xi_1)]^{-\frac{1}{2}}}{\kappa}.$$

When κ is close to zero the implementation uses $\mu = 2\xi_1 - 1$; the azimuth remains $\varphi = 2\pi\xi_2$. This sampler is paired exactly with the PDF above and requires no rejection.

Dynamic Multiple Importance Sampling

Although ALICE provides guidance that approximates the true light field extremely closely, in dynamic scenes with drastic occlusion changes, the prior guidance from the previous frame may become invalid (e.g., sudden light source movement or camera teleportation). To guarantee the absolute unbiasedness of the rendering equation and avoid division-by-zero variance explosions, we blend ALICE sampling with classical cosine-weighted sampling via Multiple Importance Sampling (MIS) [4].

Within the closure, the normalized first-moment length $\rho = \frac{|v|}{\omega}$ reflects the light field's "directional confidence." Therefore, we directly couple ALICE's mixture probability weight P_{guide} to ρ :

$$P_{\text{guide}} = \begin{cases} 0.975 \cdot \rho & \text{if } |v| > 10^{-8} \\ 0 & \text{if } |v| \leq 10^{-8} \end{cases}$$

Design Intent: When the light field tends toward diffuse ambient ($\rho \rightarrow 0$), the system automatically degenerates to cosine sampling; when the light field exhibits strong directionality ($\rho \rightarrow 1$), the system invests 97.5% of the computational effort into ALICE guidance, retaining 2.5% cosine sampling as a safety floor.

Ultimately, the mixed probability density function for the next ray is:

$$p_{\text{mix}}(\vec{u}) = (1 - P_{\text{guide}}) \cdot p_{\text{cos}}(\vec{u}) + P_{\text{guide}} \cdot p_{\text{ALICE}}(\vec{u})$$

where $p_{\text{cos}}(\vec{u}) = \frac{\max(0, \vec{n} \cdot \vec{u})}{\pi}$.

In the rendering equation's Monte Carlo integrator, the diffuse BSDF contribution (product factor) is:

$$W_{\text{BSDF}} = \frac{f_r \cdot \max(0, \vec{n} \cdot \vec{u})}{p_{\text{mix}}(\vec{u})} = \frac{p_{\text{cos}}(\vec{u})}{p_{\text{mix}}(\vec{u})}$$

The above theory ultimately converges to an extremely minimal code architecture: the real-world pipeline, without requiring any complex octree or neural grid caches, uses only the spatiotemporal filtering byproduct (v, ω) from the previous frame to accomplish, with extremely low ALU overhead, a theoretically complete and physically unbiased path guiding step.

References

- [1] C. Schied et al., "Spatiotemporal variance-guided filtering: real-time reconstruction for path-traced global illumination," in *Proceedings of High Performance Graphics*, 2017, pp. 1–12.
- [2] B. Bitterli, C. Wyman, M. Pharr, P. Shirley, A. Lefohn, and W. Jarosz, "Spatiotemporal reservoir resampling for real-time ray tracing with dynamic

direct lighting,” ACM Transactions on Graphics (TOG), vol. 39, no. 4, pp. 148–141, 2020.

- [3] NVIDIA, “NVIDIA Real-Time Denoisers (NRD).” 2021.
- [4] E. Veach and L. J. Guibas, “Optimally combining sampling techniques for Monte Carlo rendering,” in Proceedings of the 22nd annual conference on Computer graphics and interactive techniques, 1995, pp. 419–428.